



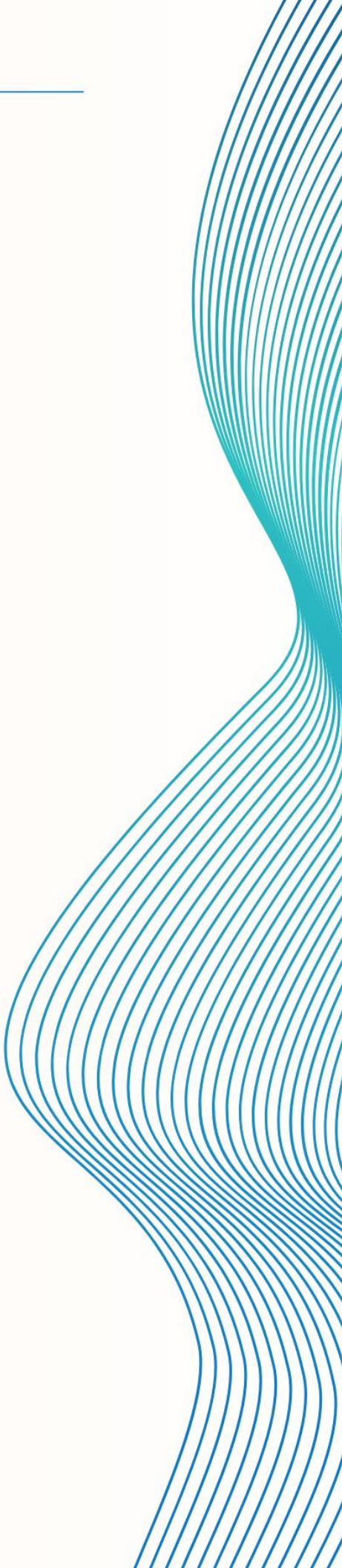
DISEÑO DEL **SOFTWARE**

Sistema ANPR Vision

Versión 1.0 – 2025

Presentado por:

Equipo de Desarrollo ANPR Vision



Contenido

- 1. Introducción
 - 1.1 Propósito del documento
- 2. Modelo funcional
 - 2.1 Casos de uso
 - 2. Historias de usuario
 - Administrador
 - Usuario Final
- 3. Modelado del sistema
 - Arquitectura General del Sistema
 - 4.2 Diagrama Entidad-Relación (MER)
 - Diagrama de Clases
 - Diagrama de Flujo
 - Diagramas de secuencia

1. Introducción

1.1 Propósito del documento

El presente documento describe el diseño de software del sistema **ANPR-VISION**, un sistema de gestión de parqueaderos con reconocimiento automático de matrículas (ANPR).

El objetivo es definir, de forma clara y sintética, los artefactos de diseño que guían la implementación:

- Casos de uso y su comportamiento principal.
- Historias de usuario.
- Modelo estático (diagrama de clases y MER).
- Modelo dinámico (flujos y secuencias).

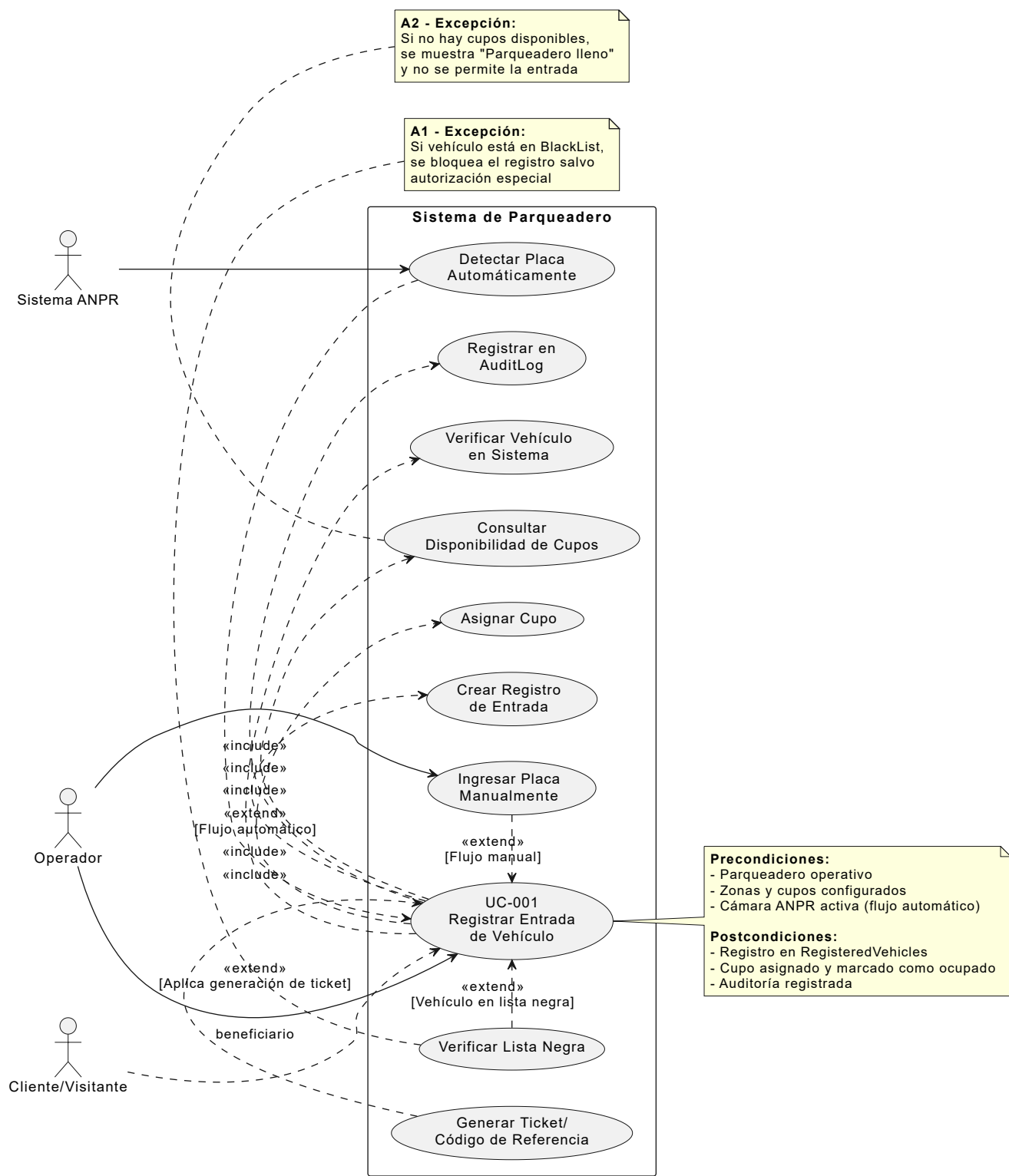
2. Modelo funcional

2.1 Casos de uso

En esta sección se presentan los **casos de uso principales** identificados para ANPR-VISION.

Campo	Detalle
Código	UC-001
Nombre	Registrar Entrada de Vehículo
Actor principal	Sistema / Operador
Descripción	Permite registrar la entrada de un vehículo al parqueadero, ya sea a partir de la detección automática de la placa (ANPR) o mediante registro manual por parte del operador.
Tipo	Primario
Precondiciones	- El parqueadero está operativo y configurado. - Existen zonas, sectores y cupos configurados. - La cámara ANPR asociada al acceso está registrada y activa (para flujo automático).
Postcondiciones	- Se crea un registro de entrada en RegisteredVehicles con estado Active. - Se asocia el vehículo a un cupo - El cupo queda marcado como no disponible.
Disparador	- Captura de placa en el punto de ingreso o - Operador selecciona "Registrar Entrada" y digita la placa.

Campo	Detalle
Flujo básico	1. El sistema recibe la placa detectada o ingresada. 2. El sistema verifica si el vehículo existe; si no, crea el vehículo asociado a un cliente (o lo marca como visitante). 3. El sistema consulta disponibilidad de cupos según tipo de vehículo y reglas definidas. 4. El sistema selecciona el cupo sugerido. 5. El sistema crea el registro en RegisteredVehicles (EntryDate = fecha/hora actual, Status = Active). 6. El sistema marca el cupo como ocupado. 7. El sistema muestra confirmación al operador y, si aplica, genera ticket/recibo o código de referencia.
Flujos alternos / Excepciones	A1 – Vehículo en lista negra: el sistema detecta que la placa está en BlackList → muestra alerta y no permite registrar la entrada, salvo autorización especial. A2 – Sin cupos disponibles: no hay cupos para el tipo de vehículo → el sistema muestra mensaje de parqueadero lleno y no registra la entrada. A3 – Error en lectura de placa: la cámara no reconoce la placa → el operador puede corregir o ingresar la placa manualmente.
Requisitos especiales	- Tiempo de respuesta bajo (≤ 2–3 s desde la detección hasta la confirmación). - Trazabilidad mediante AuditLog de quién registró la entrada (operador/sistema).



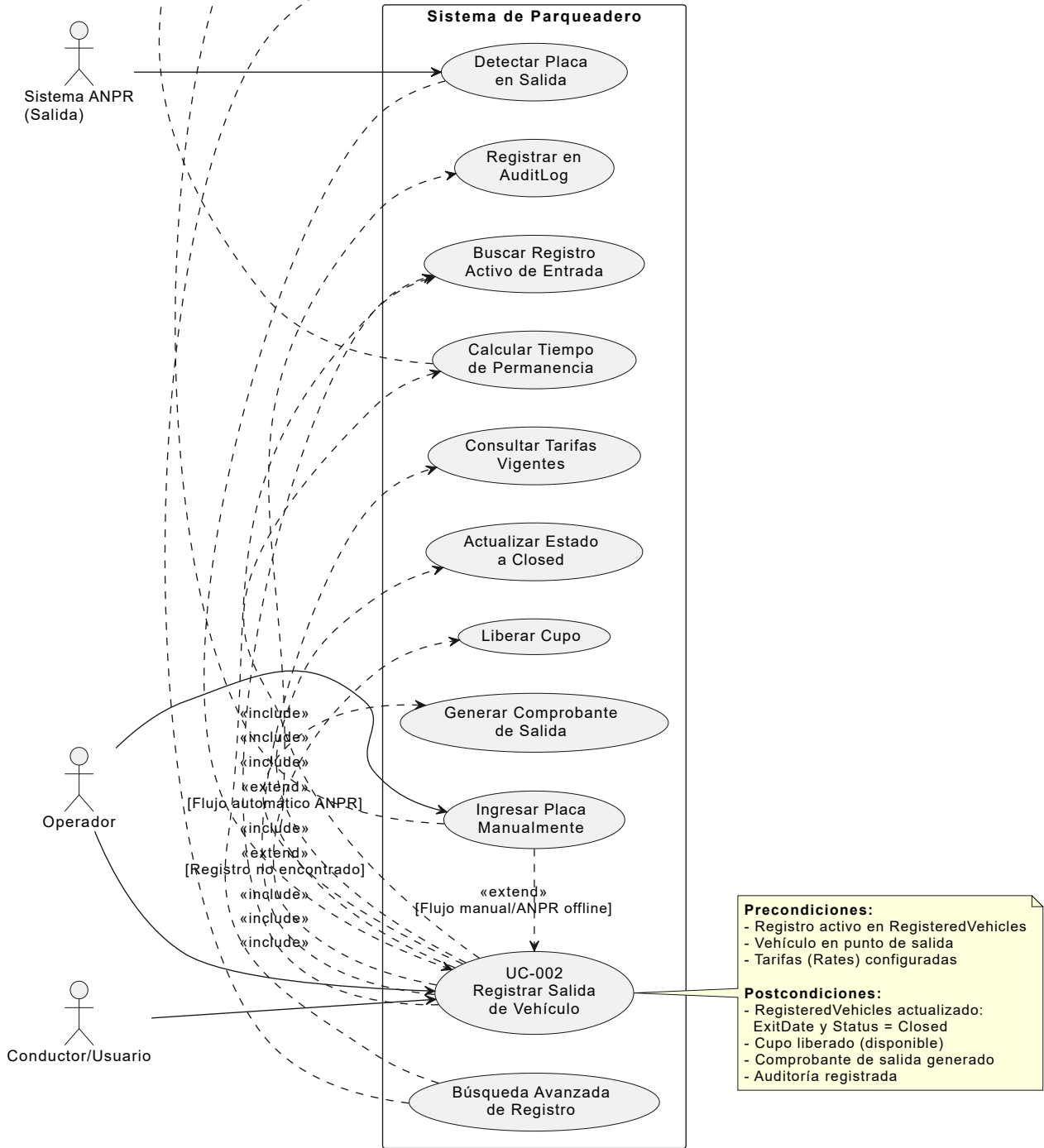
Campo	Detalle
Código	UC-002
Nombre	Registrar Salida de Vehículo
Actor principal	Conductor / Usuario (a través del sistema)
Descripción	Permite registrar la salida de un vehículo y liberar el cupo ocupado.
Tipo	Primario
Precondiciones	- Existe un registro activo en RegisteredVehicles para la placa. - El vehículo se encuentra físicamente en el punto de salida.

Campo	Detalle
	- Las tarifas (Rates) están configuradas para el parqueadero y tipo de vehículo.
Postcondiciones	- El registro en RegisteredVehicles se actualiza con ExitDate y Status = Closed. - El cupo asociado se marca como disponible. - Se genera ticket y registro de salida.
Disparador	La cámara de salida detecta la placa o el operador / usuario solicita la salida ingresando la placa.
Flujo básico	1. El sistema recibe la placa desde la cámara o el operador. 2. El sistema busca el registro activo de entrada (RegisteredVehicles). 3. El sistema calcula el tiempo de permanencia y consulta las tarifas vigentes (Rates). 7. El sistema libera el cupo asociado. 8. El sistema genera el comprobante o registro de salida.
Flujos alternos / Excepciones	A1 – Registro no encontrado: no existe entrada activa para la placa → el sistema muestra error y permite búsqueda avanzada (por fecha, zona, etc.).
Requisitos especiales	- Cálculo de tarifa confiable y auditable. - Debe poder operar incluso si el módulo ANPR está temporalmente fuera de línea (entrada manual).

Requisito especial:
Cálculo de tarifa confiable y auditable basado en:
- Tiempo de permanencia
- Tarifas vigentes (Rates)
- Tipo de vehículo

Requisito especial:
Sistema debe operar incluso si módulo ANPR está offline (entrada manual como respaldo)

A1 - Excepción:
Si no existe entrada activa para la placa, se permite búsqueda por fecha, zona, sector, etc.



Campo	Detalle
Código	UC-003
Nombre	Detectar Placa
Actor principal	Cámara ANPR (microservicio de visión)
Descripción	El microservicio de visión captura imágenes del stream de la cámara, detecta la matrícula del vehículo, la normaliza y la envía al backend para su procesamiento.
Tipo	Soporte / Técnico
Precondiciones	- Cámara configurada y asociada a un Parking y a un acceso (entrada/salida). - Microservicio de ANPR en ejecución y conectado a Kafka/backend.
Postcondiciones	- Se genera un evento de detección de placa con metadatos (placa, cámara, hora, confianza, tipo de acceso) enviado al backend. - Opcionalmente se genera una imagen recortada de la placa o del vehículo.
Disparador	Nueva imagen/frame disponible en el stream de video de la cámara.
Flujo básico	1. El microservicio lee el frame de la cámara asociada. 2. Aplica el modelo de detección (YOLO u otro) para localizar la placa. 3. Aplica OCR para extraer el texto de la matrícula. 4. Normaliza la placa (mayúsculas, sin espacios, formato país). 5. Encola/produce un mensaje en Kafka con la detección y datos de contexto. 6. El backend consume el evento para los casos de uso UC-001 / UC-002 / UC-004.
Flujos alternos / Excepciones	A1 – Placa no legible: el OCR no logra confianza mínima → se marca detección como “no confiable” y se envía para revisión manual. A2 – Cámara fuera de servicio: no se recibe video → el microservicio registra evento de error y notifica al backend.
Requisitos especiales	- Latencia de detección baja (ideal < 1s por frame procesado). - Tasa de error controlada (definida en requisitos no funcionales).

Normalización:

- Mayúsculas
- Sin espacios
- Formato según país

Modelo de detección:
YOLO u otro modelo de deep learning para localización de placas

Requisitos especiales:

- Latencia < 1s por frame
- Tasa de error controlada
- Mensaje incluye metadatos: placa, cámara, hora, confianza

A2 - Excepción:
Si no se recibe video de cámara, se registra evento de error y se notifica al backend sobre cámara fuera de servicio

A1 - Excepción:
Si OCR no alcanza confianza mínima, se marca como "no confiable" y se envía para revisión manual

Cámara ANPR

stream de video

Microservicio de Visión

Backend Sistema

Módulo de Detección ANPR

Leer Frame de Cámara

Localizar Placa en Imagen

Aplicar OCR para Extraer Texto

Generar Evento de Detección

Normalizar Formato de Placa

Aplicar Modelo de Detección (YOLO)

«include»

UC-003 Detectar Placa

«extend»
[Opcional: recorte de placa]

Generar Imagen Recortada

Publicar Mensaje en Kafka

Marcar Detección

«include»

«include»

«include»

consume evento

«extend»
[Cámara fuera de servicio]

«extend»
[OCR < confianza mínima]

«include»

«include»

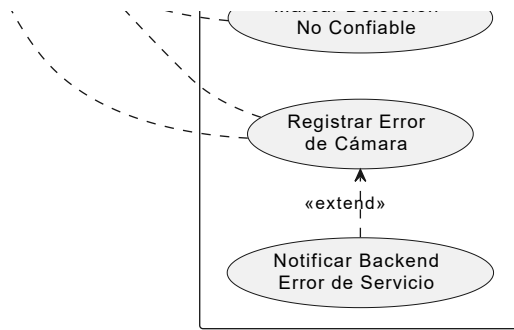
«include»

Precondiciones:

- Cámara configurada y asociada a Parking y tipo de acceso
- Microservicio ANPR en ejecución
- Conexión con Kafka/backend activa

Postcondiciones:

- Evento de detección generado con:
 - Placa detectada
 - ID de cámara
 - Timestamp
 - Nivel de confianza
 - Tipo de acceso (entrada/salida)
- Imagen recortada (opcional)



Campo	Detalle
Código	UC-004
Nombre	Notificar Detección al Operador
Actor principal	Sistema Backend
Descripción	El backend recibe un evento de detección de placa y genera una notificación en tiempo real al operador para que visualice y confirme la acción correspondiente (entrada, salida, alerta, etc.).
Tipo	Secundario / Soporte
Precondiciones	- El backend está suscrito a los eventos de Kafka (detección). - El operador está autenticado y conectado al portal (SignalR activo).
Postcondiciones	- Se crea un registro en Notification asociado al Parking correspondiente. - La notificación se envía en tiempo real al cliente web/móvil del operador.
Disparador	Recepción de un evento de detección de placa desde el microservicio (UC-003).
Flujo básico	1. El backend consume el evento de detección desde Kafka. 2. Valida la estructura y datos básicos (placa, cámara, parking). 3. Opcionalmente, consulta si el vehículo tiene registro activo o membresía. 4. Construye un mensaje amigable para el operador (ej. "Vehículo ABC123 detectado en entrada principal"). 5. Crea la entidad Notification en la base de datos. 6. Usa SignalR para enviar la notificación al grupo de operadores del Parking correspondiente. 7. El operador visualiza la notificación en el panel.
Flujos alternos / Excepciones	A1 – Error en persistencia: falla al guardar la notificación → el sistema registra el error en AuditLog / logs y reintenta o marca como notificación no persistida. A2 – Operador desconectado: si no hay conexiones activas, la notificación quedará pendiente y se mostrará la próxima vez que el operador consulte la bandeja.
Requisitos especiales	- Entrega casi en tiempo real (latencia baja). - Garantía de no pérdida de notificaciones críticas (política de reintentos / almacenamiento).

Requisitos especiales:

- Latencia baja (tiempo real)
- Envío al grupo de operadores del Parking específico
- Garantía de no pérdida de notificaciones críticas

Ejemplo de mensaje:
"Vehículo ABC123 detectado en entrada principal"

Incluye:

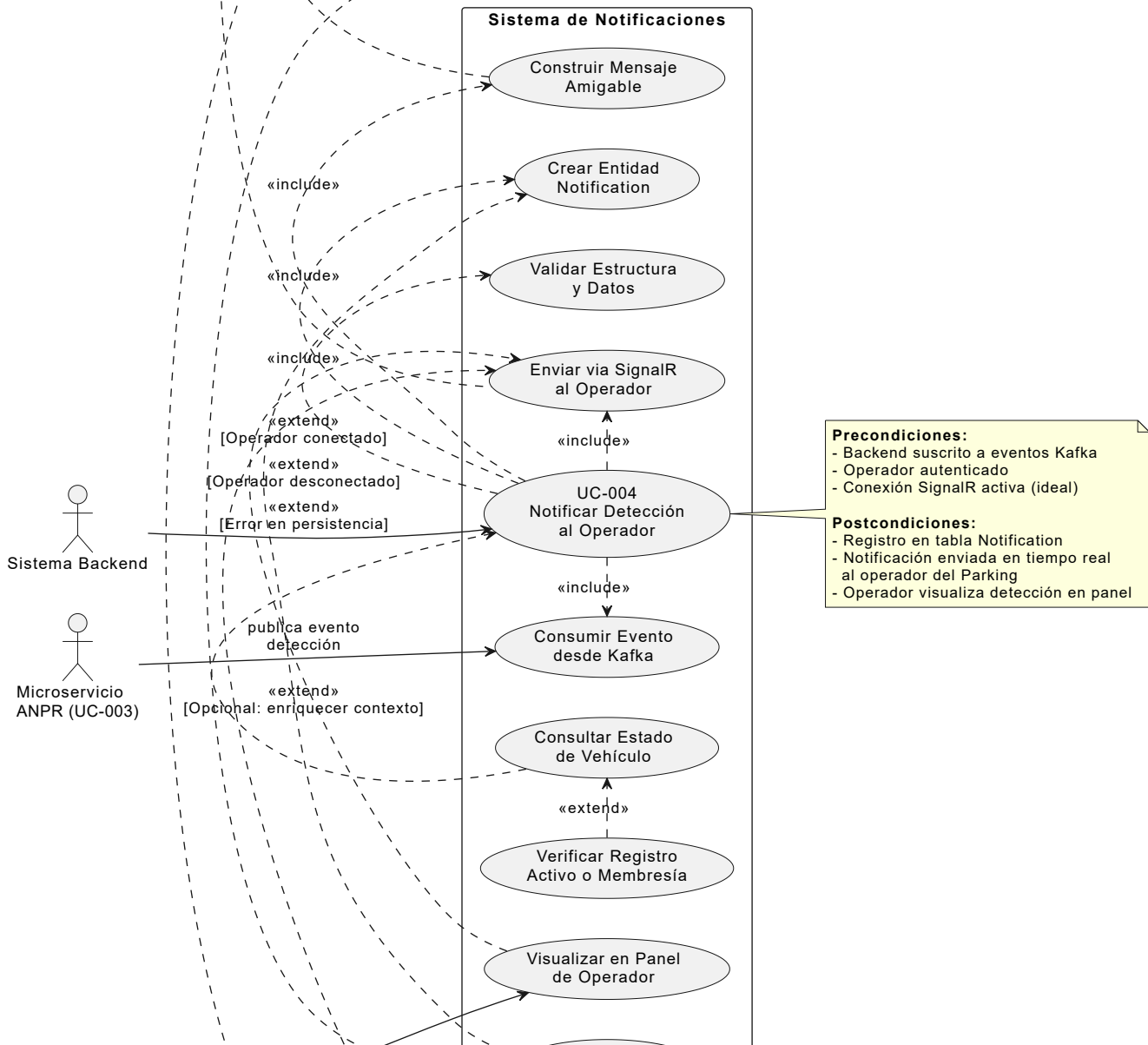
- Placa normalizada
- Punto de acceso
- Timestamp
- Contexto adicional (opcional)

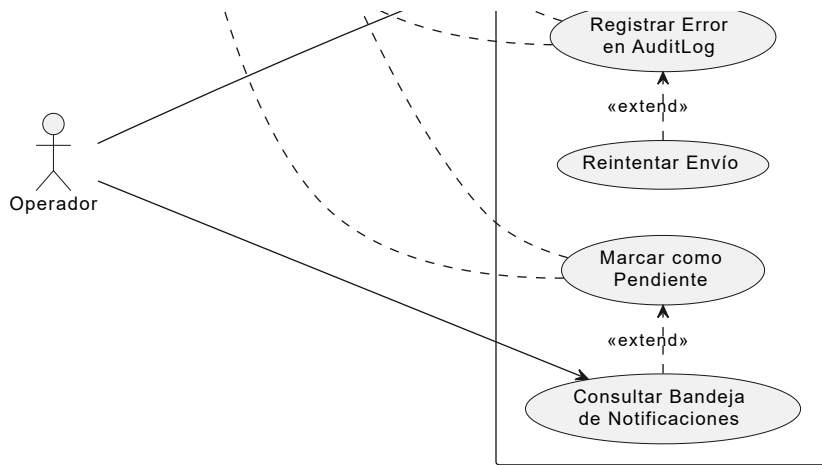
A2 - Excepción:
Si operador desconectado:

- Notificación queda pendiente
- Se mostrará cuando operador consulte bandeja de notificaciones
- No se pierde la notificación

A1 - Excepción:
Si falla persistencia de notificación:

- Registrar error en AuditLog
- Aplicar política de reintentos
- Marcar como no persistida si reintentos fallan





Campo	Detalle
Código	UC-005
Nombre	Consultar Disponibilidad de Cupos
Actor principal	Operador / Sistema
Descripción	Permite conocer en tiempo real la ocupación del parqueadero (por zonas, sectores, tipos de vehículo) para apoyar la toma de decisiones y la asignación de cupos.
Tipo	Primario (informativo/consulta)
Precondiciones	- Zonas, sectores y cupos están configurados. - Existen registros en RegisteredVehicles y Slots actualizados.
Postcondiciones	- Se muestra al actor un resumen de disponibilidad (cupos libres, ocupados, porcentajes) por parking, zona y tipo de vehículo.
Disparador	El operador abre el panel de ocupación o el sistema requiere validar disponibilidad para UC-001.
Flujo básico	1. El actor solicita la vista de ocupación (dashboard o reporte). 2. El sistema consulta Slots, Sectores, Zonas y registros activos en RegisteredVehicles. 3. Calcula cupos ocupados y libres por cada sector/zona/tipo. 4. Muestra los resultados en formato de tablas, gráficos o indicadores (ej. "80% ocupado").
Flujos alternos / Excepciones	A1 – Sin datos de sectores/cupos: si no hay configuración, el sistema muestra mensaje de advertencia y sugiere configurar estructura del parking. A2 – Error de conexión a base de datos: se muestra mensaje de error y se registra en AuditLog.
Requisitos especiales	- Debe obtener información casi en tiempo real. - Debe poder filtrarse por rango de fechas (historial) en versiones avanzadas.

Requisito especial:
Información en tiempo real:
- Consulta optimizada
- Caché cuando sea apropiado
- Actualización periódica

Requisito especial:
Versión avanzada permite consultar historial de ocupación por rango de fechas

Formato de resumen:
- Tablas por zona/sector
- Gráficos de ocupación
- Indicadores visuales (ej. "80% ocupado")
- Desglose por tipo de vehículo

A2 - Excepción:
Si error de conexión a BD:
- Mostrar mensaje de error
- Registrar en AuditLog
- Intentar recuperación automática

A1 - Excepción:
Si no existe configuración de sectores/cupos:
- Mostrar mensaje de advertencia
- Sugerir configurar estructura del parking

Sistema de Gestión de Parquadero

Consultar Slots y Sectores

Consultar Zonas Configuradas

Consultar Registros Activos

Calcular Ocupación por Sector/Zona

Calcular Disponibilidad por Tipo Vehículo

Generar Resumen Estadístico

Solicitar Vista de Ocupación

UC-005
Consultar Disponibilidad de Cupos

Filtrar por

«include»

«include»

«include»

«include»

«include»

«extend»
[Vista gráfica]

«extend»
[Sin configuración]

«extend»
[Error conexión BD]

«include»

«extend»
[Sistema valida para UC-001]

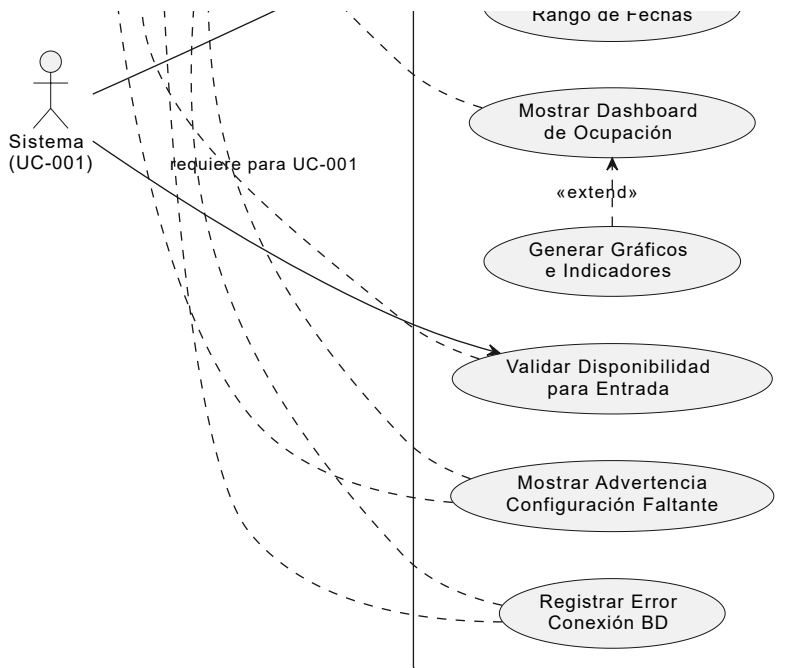
«extend»
[Operador consulta manual]

«extend»
[Consulta histórica]

Precondiciones:
- Zonas, sectores y cupos configurados
- Registros en RegisteredVehicles
- Tabla Slots actualizada

Postcondiciones:
- Resumen de disponibilidad generado:
• Cupos libres por zona/sector
• Cupos ocupados por zona/sector
• Porcentaje de ocupación
• Distribución por tipo de vehículo





2. Historias de usuario

Las historias de usuario complementan los casos de uso, desde la perspectiva ágil.

Administrador

- HU01. Como **administrador**, quiero **iniciar sesión en el sistema** para **gestionar la plataforma de forma segura**.
- HU02. Como **administrador**, quiero **visualizar en un dashboard los vehículos activos, alertas e informes gráficos** para **tener control del parqueadero en tiempo real**.
- HU03. Como **administrador**, quiero **visualizar la lista negra de vehículos** para **evitar el acceso no autorizado**.
- HU04. Como **administrador**, quiero **ver reportes del uso del parqueadero y estadísticas** para **tomar decisiones basadas en datos**.
- HU05. Como **administrador**, quiero **asignar roles y permisos al personal** para **controlar el acceso a las funcionalidades del sistema**.
- HU08. Como **administrador**, quiero **ver cuántos cupos están disponibles** para **gestionar el ingreso de vehículos**.
- HU09. Como **administrador**, quiero **visualizar y gestionar las alertas de infracción** para **notificar al cliente o registrar eventos importantes**.
- HU11. Como **administrador**, quiero **consultar el historial de un vehículo** para **ver entradas, salidas**.
- HU12. Como **administrador**, quiero **registrar manualmente un vehículo si falla el reconocimiento automático** para **no interrumpir el servicio**.

Usuario Final

- HU13. Como **usuario final**, quiero **consultar la disponibilidad de espacios** para **decidir si ingreso o busco otro parqueadero**.
- HU14. Como **usuario final**, quiero **ingresar al sistema con mi placa y documento** para **ver información relacionada a mi vehículo**.

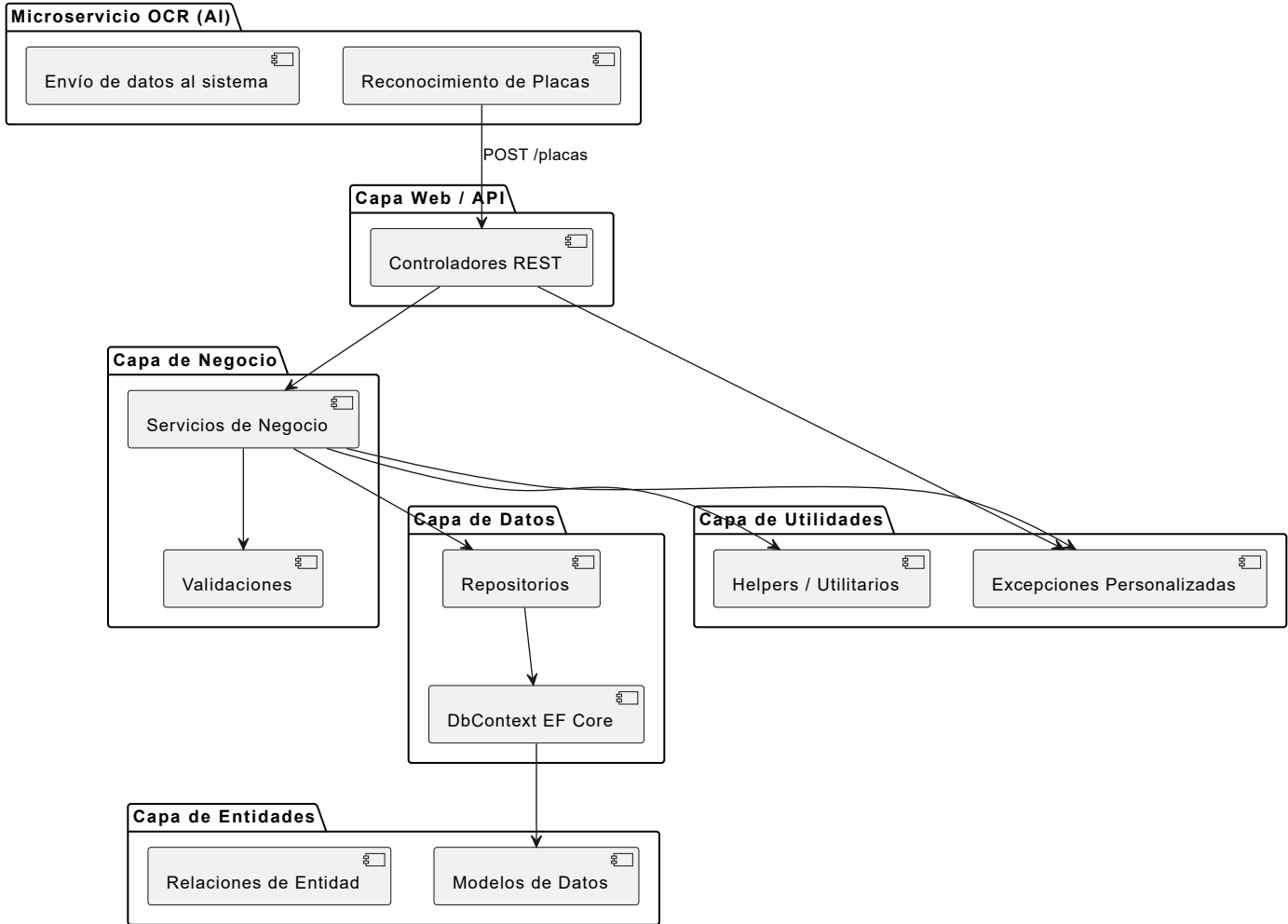
3. Modelado del sistema

Arquitectura General del Sistema

El sistema está desarrollado bajo una arquitectura multicapa basada en el patrón **N-Capas**, dividiendo claramente las responsabilidades de cada módulo:

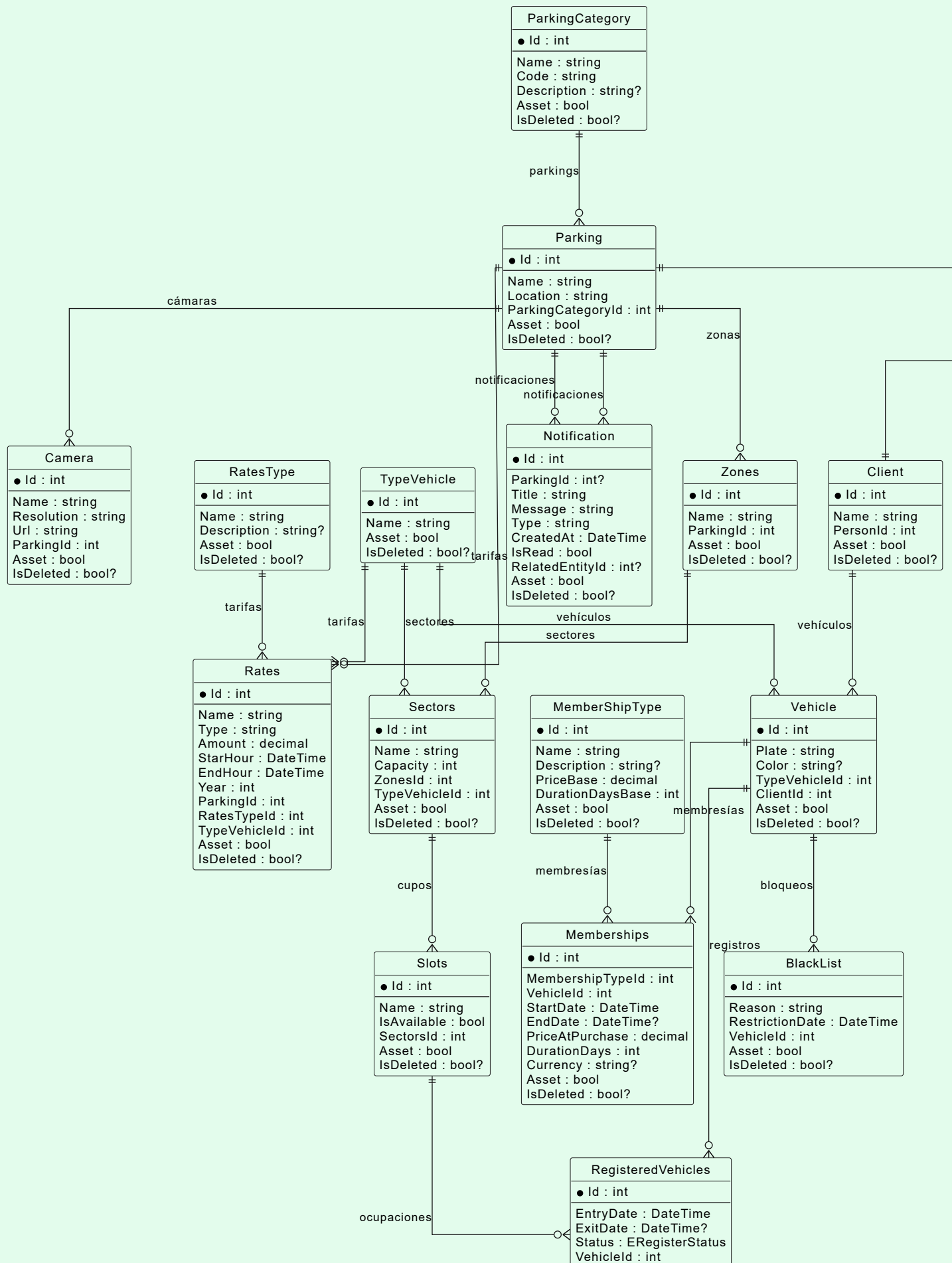
- **Capa de Entidades (Entity):** Define los modelos de datos y relaciones entre las entidades del sistema.
- **Capa de Datos (Data):** Encargada del acceso a la base de datos, utilizando Entity Framework Core.
- **Capa de Negocio (Business):** Contiene la lógica del negocio y las validaciones de las operaciones principales.
- **Capa de Utilidades (Utilities):** Define excepciones personalizadas y otras clases de apoyo comunes.
- **Capa Web/API (Web):** Provee la interfaz para el consumo del sistema a través de servicios RESTful.
- **Capa OCR (Microservicio externo):** Módulo encargado del reconocimiento de matrículas mediante inteligencia artificial y envío de datos al sistema.

Arquitectura General del Sistema - ANPR VISION



4.2 Diagrama Entidad-Relación (MER)

Módulo Operación Parking



	SlotsId : int? Asset : bool IsDeleted : bool?	
--	---	--

Diagrama de Clases

C

Camera

○ Resolution : string

○ Url : string

○ ParkingId : int

● ExistsAsync(predicate : Expression<Func<Camera, bool>>) : Task<bool>

● GetAll(filters : IDictionary<string, string?>?) : Task<List<CameraDto>>

● GetById(id : int) : Task<CameraDto>

● Save(entityDto : CameraDto) : Task<CameraDto>

● Update(entityDto : CameraDto) : Task

● Delete(id : int) : Task<int>

C

ParkingCategory

○ Code : string

○ Description : string?

● ExistsAsync(predicate : Expression<Func<ParkingCategory, bool>>) : Task<bool>

● GetAll(filters : IDictionary<string, string?>?) : Task<List<ParkingCategoryDto>>

● GetById(id : int) : Task<ParkingCategoryDto>

● Save(entityDto : ParkingCategoryDto) : Task<ParkingCategoryDto>

● Update(entityDto : ParkingCategoryDto) : Task

● Delete(id : int) : Task<int>

C

BlackList

○ Reason : string

○ RestrictionDate : DateTime

○ VehicleId : int

● ExistsAsync(predicate : Expression<Func<BlackList, bool>>) : Task<bool>

● GetAll(filters : IDictionary<string, string?>?) : Task<List<BlackListDto>>

● GetById(id : int) : Task<BlackListDto>

● Save(entityDto : BlackListDto) : Task<BlackListDto>

● Update(entityDto : BlackListDto) : Task

● Delete(id : int) : Task<int>

○ EntryC

○ ExitDa

○ Status

○ Vehicle

○ Slots

● GetAll

● GetTo

● GetTo

● GetVe

● GetSe

● GetBy

● Regist

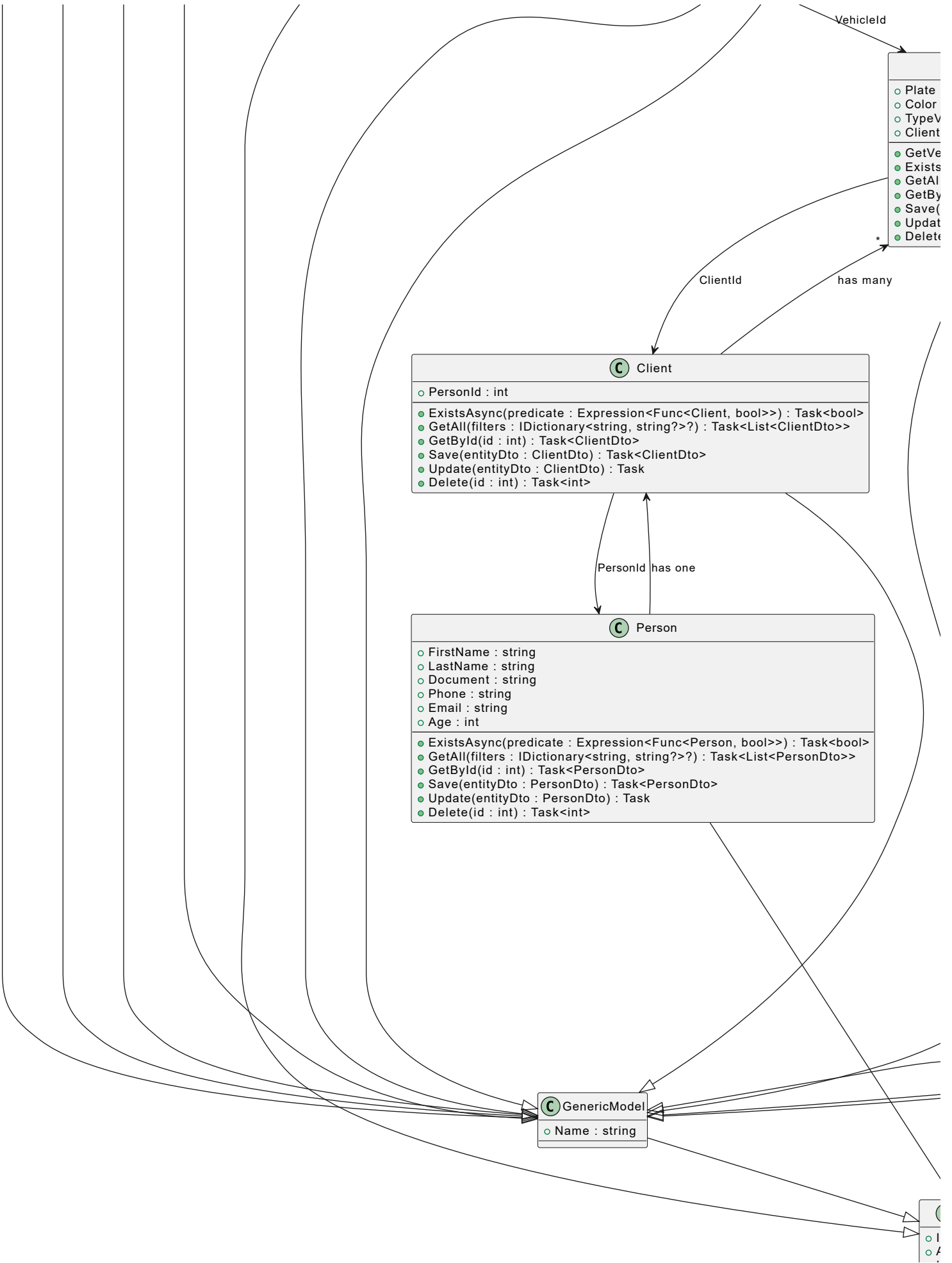
● Regist

● Manua

● Valida

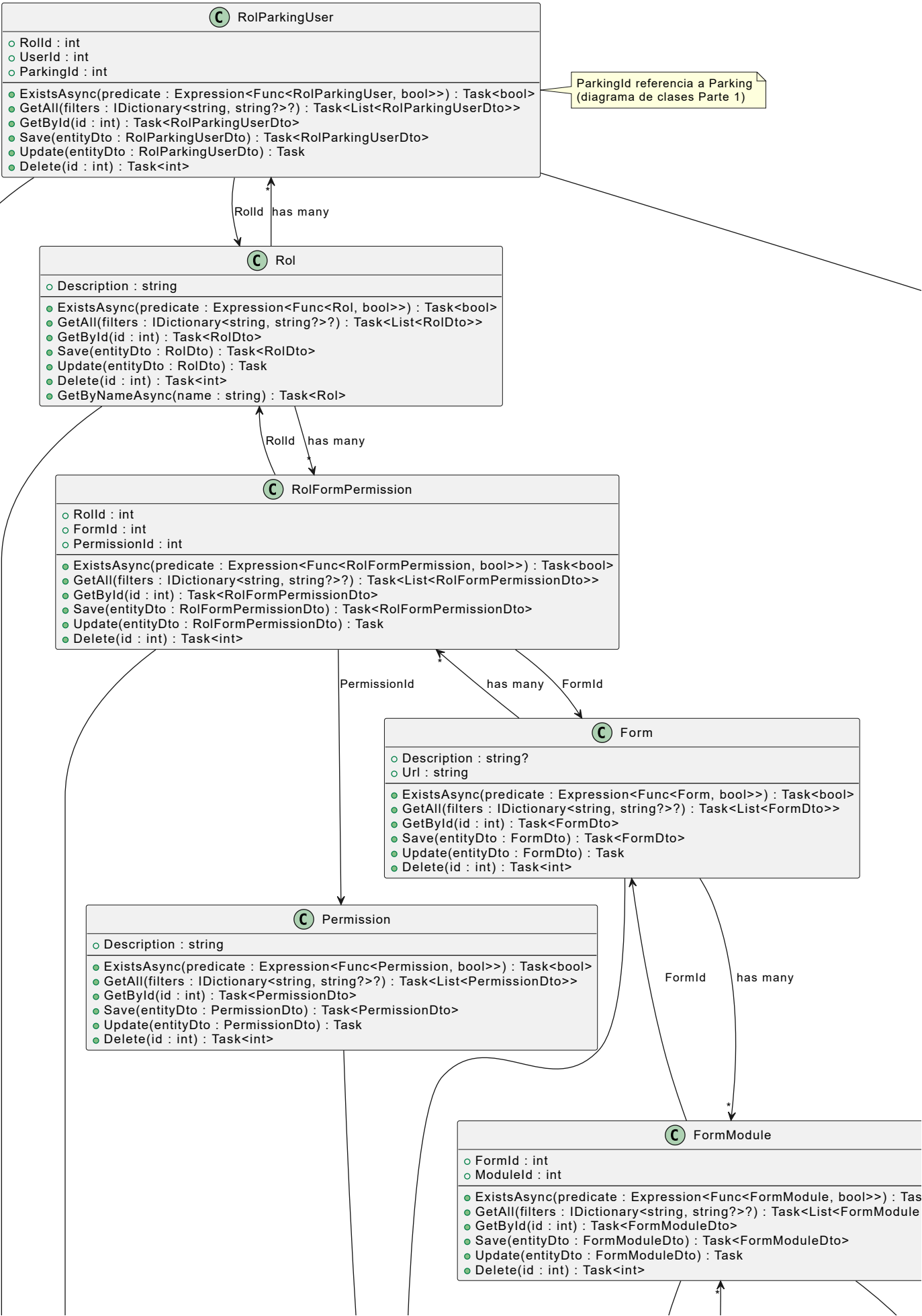
● GetRe

ParkingId has many





parte 2:



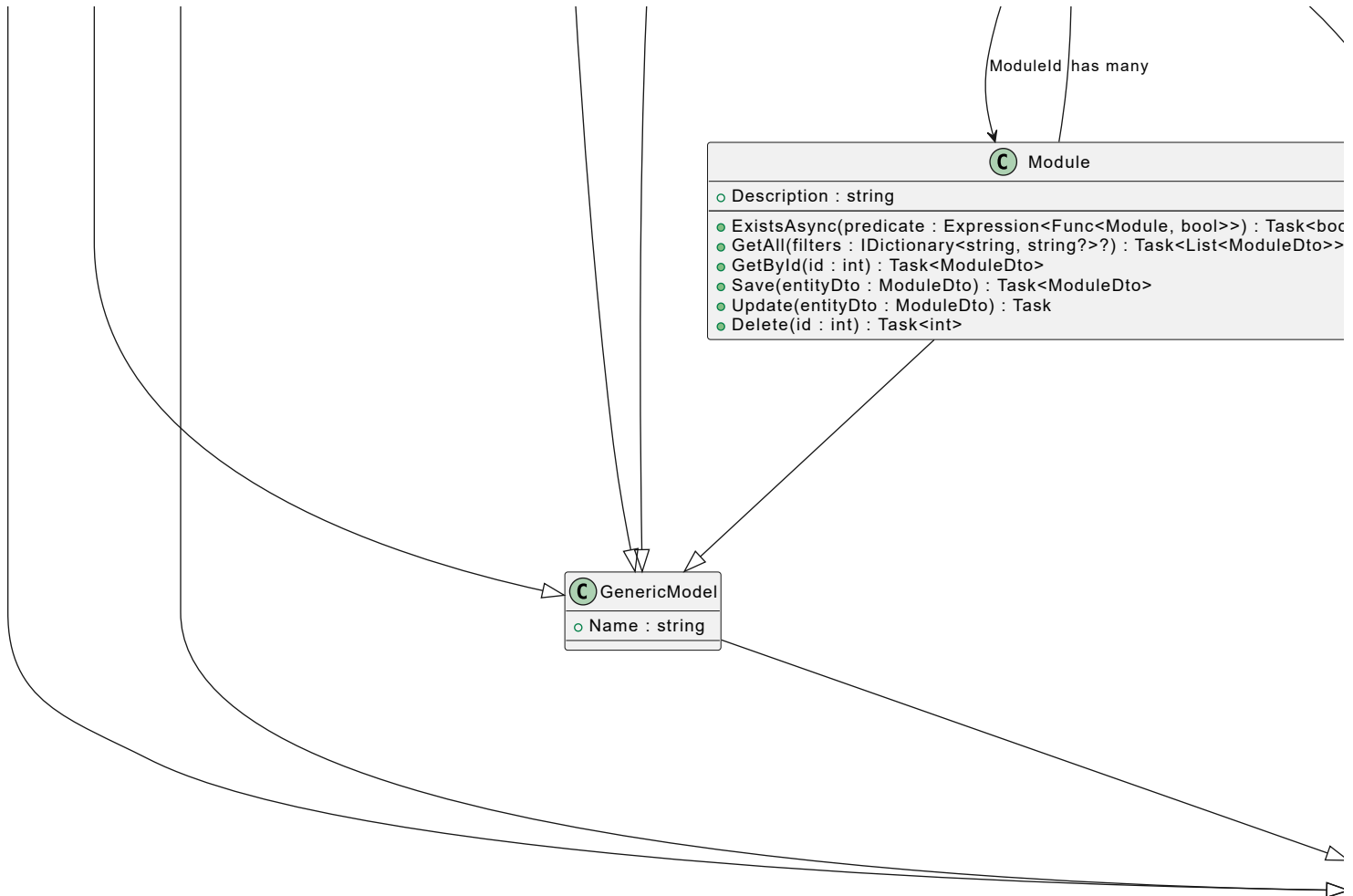
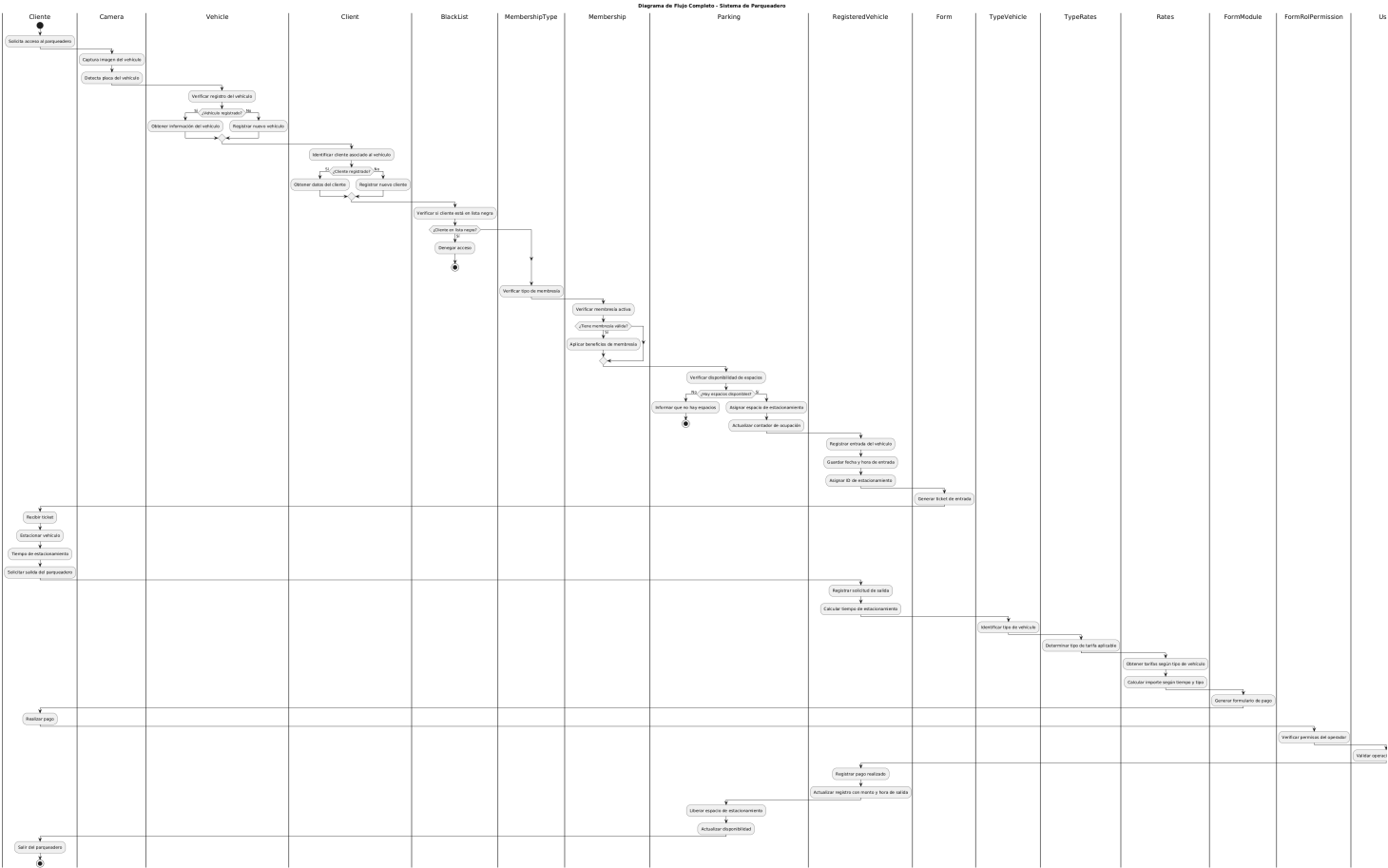
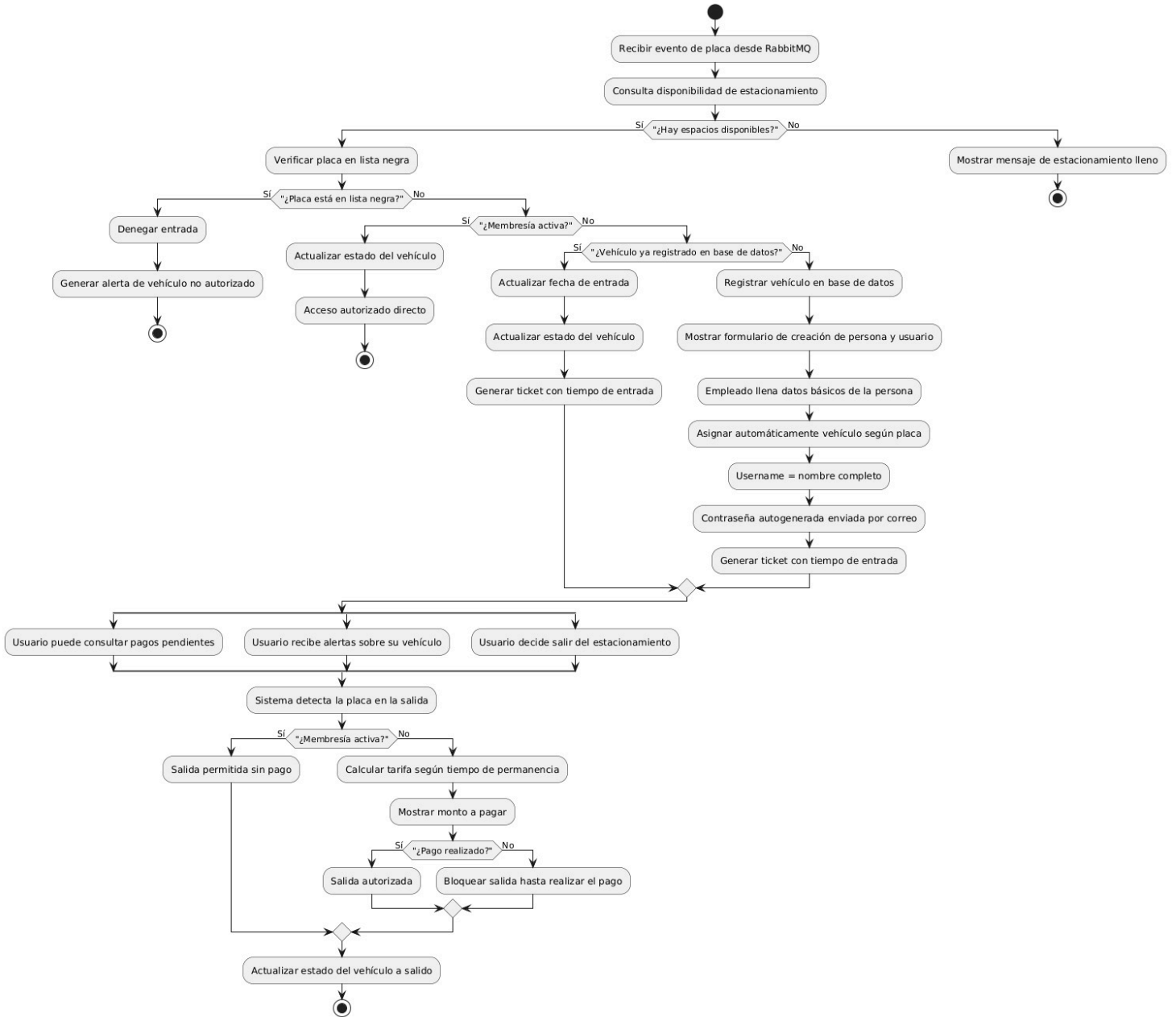
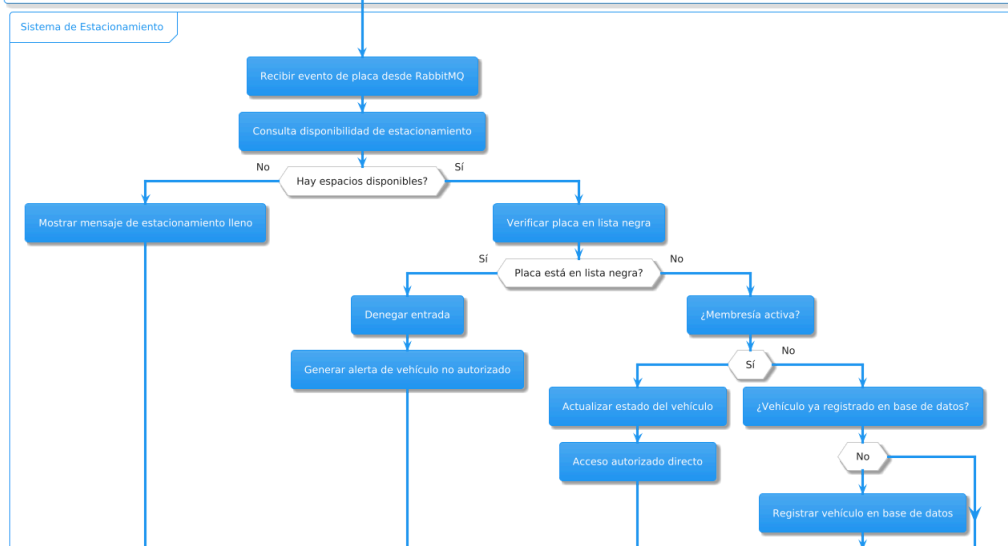
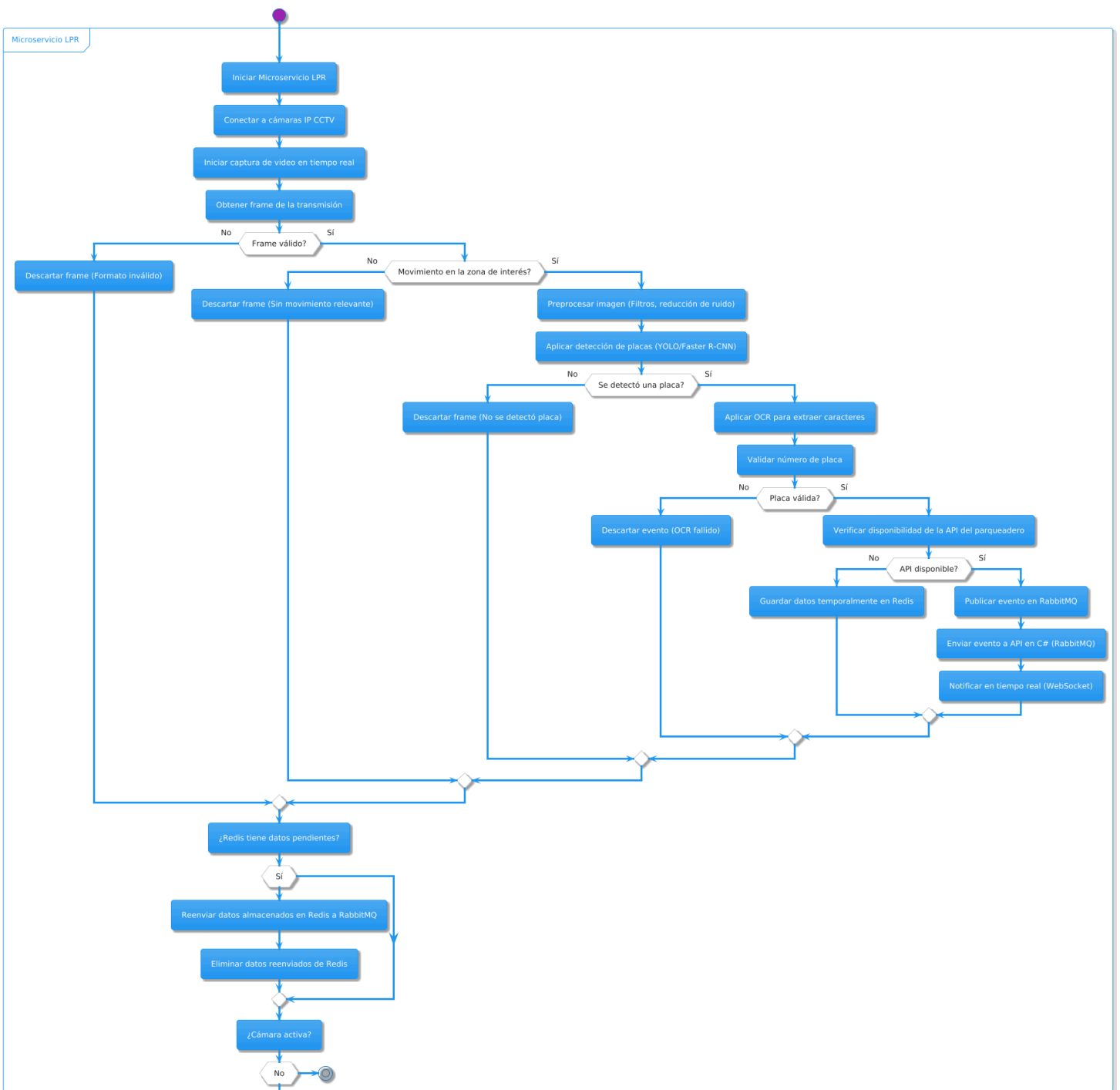


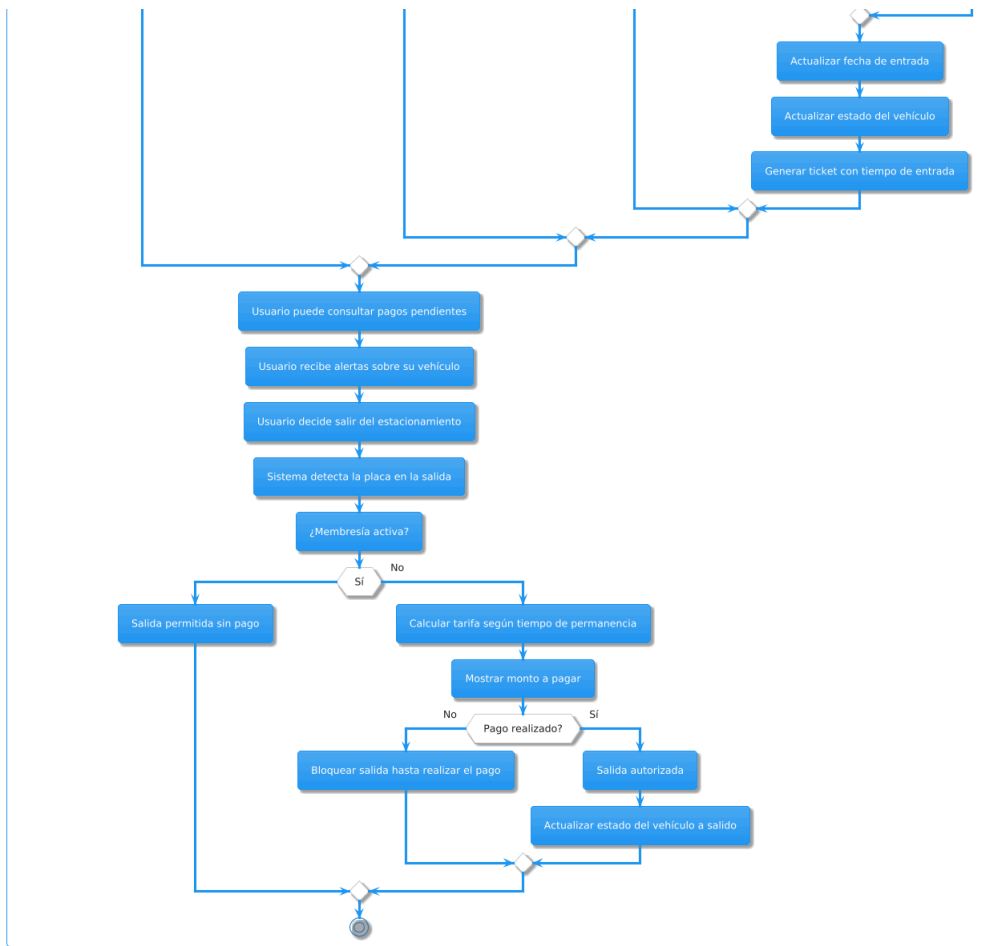
Diagrama de Flujo



Sistema de Estacionamiento

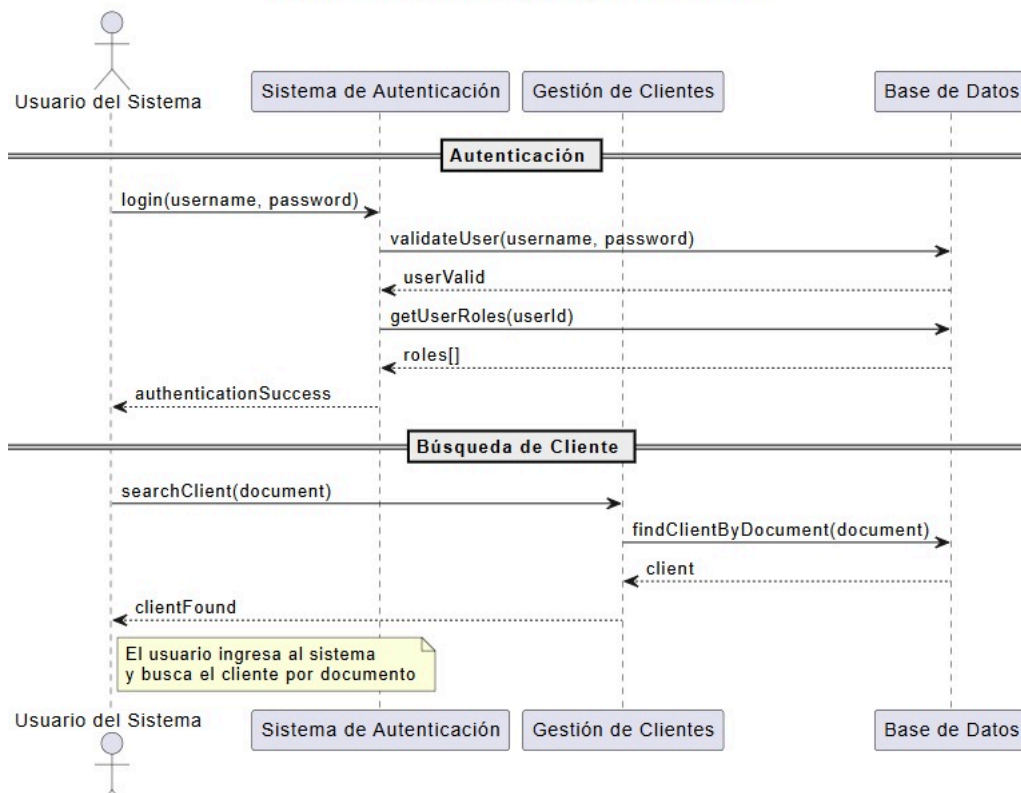




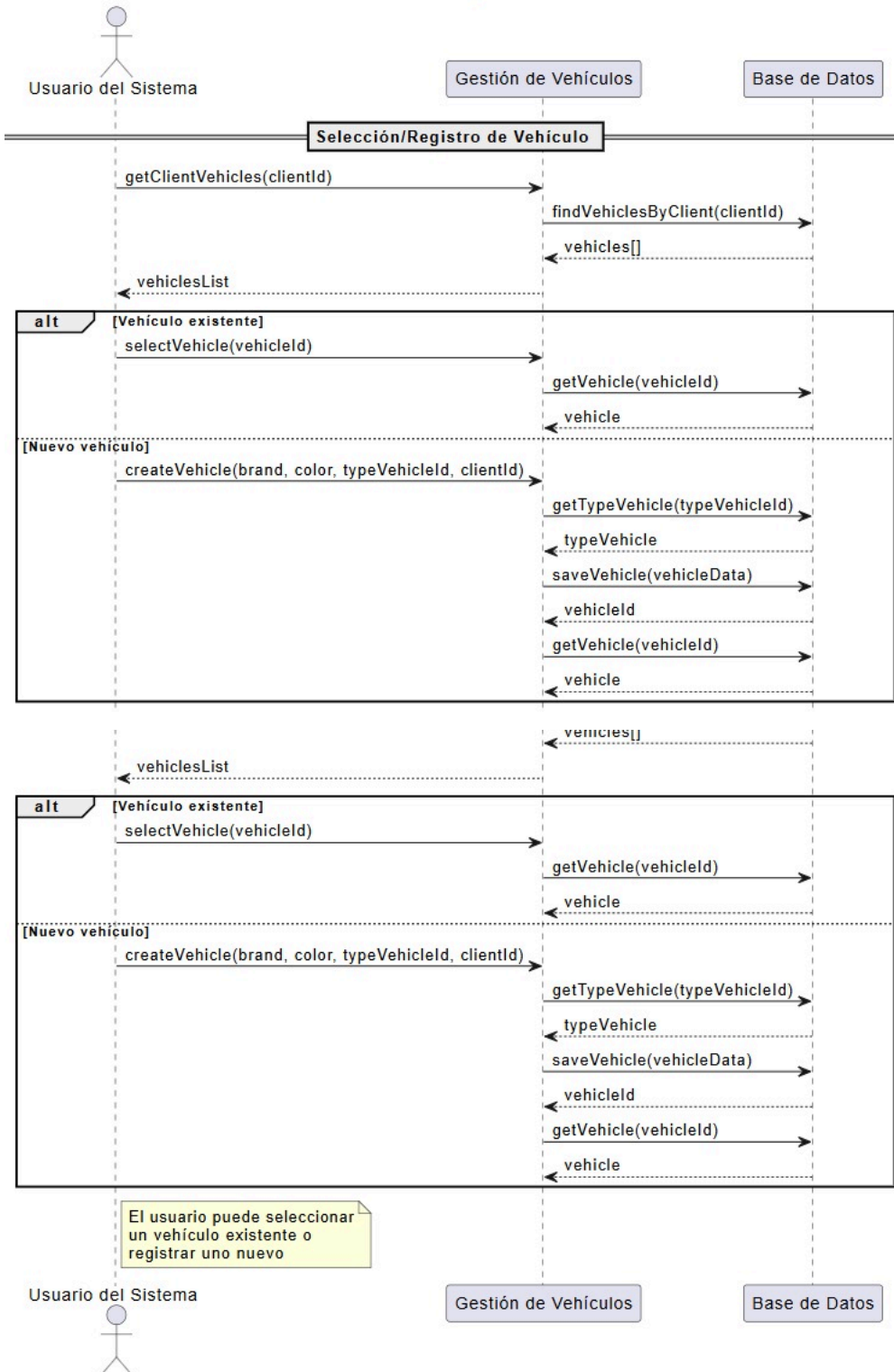


Diagramas de secuencia

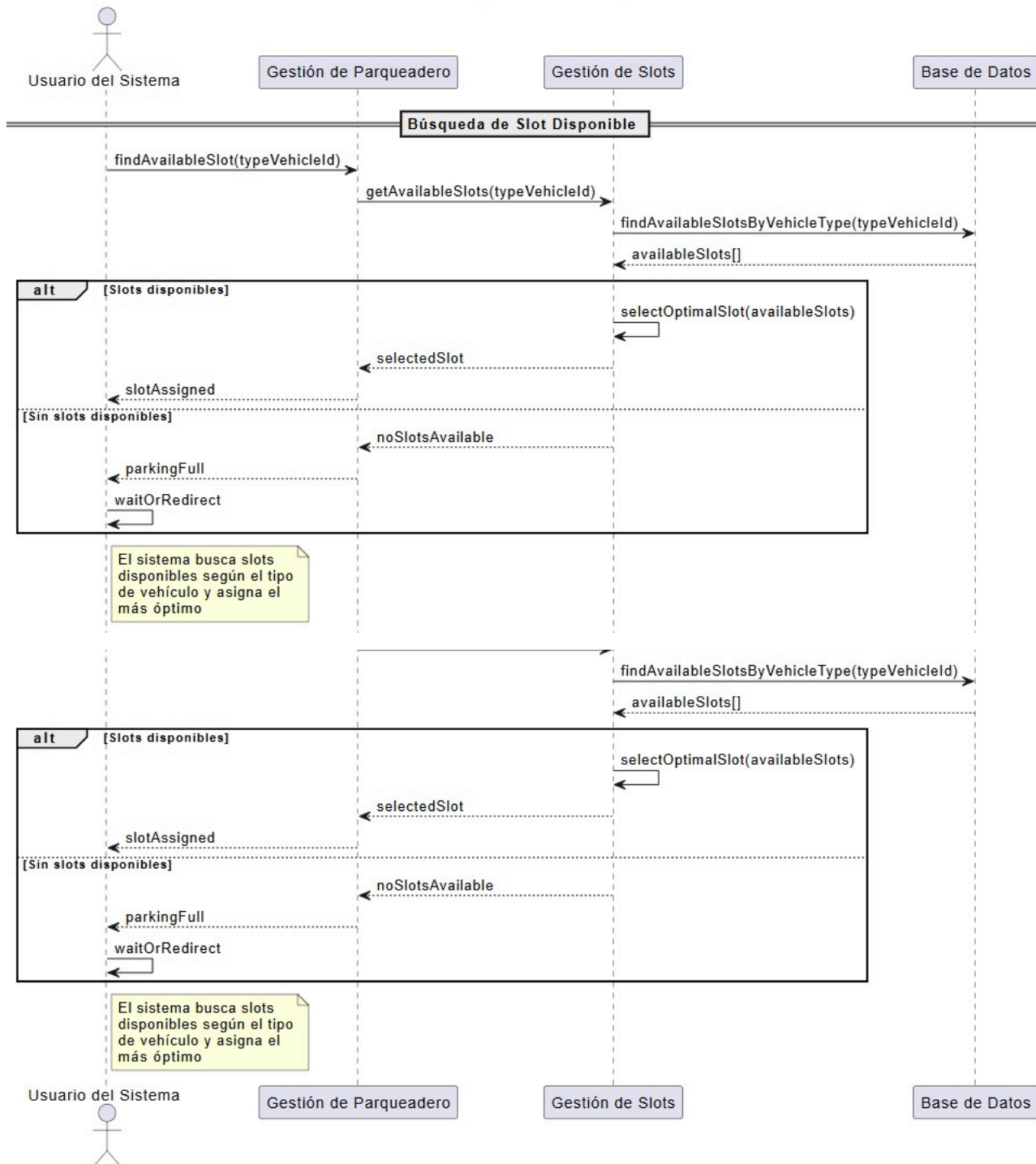
Parte 1: Autenticación y Búsqueda de Cliente



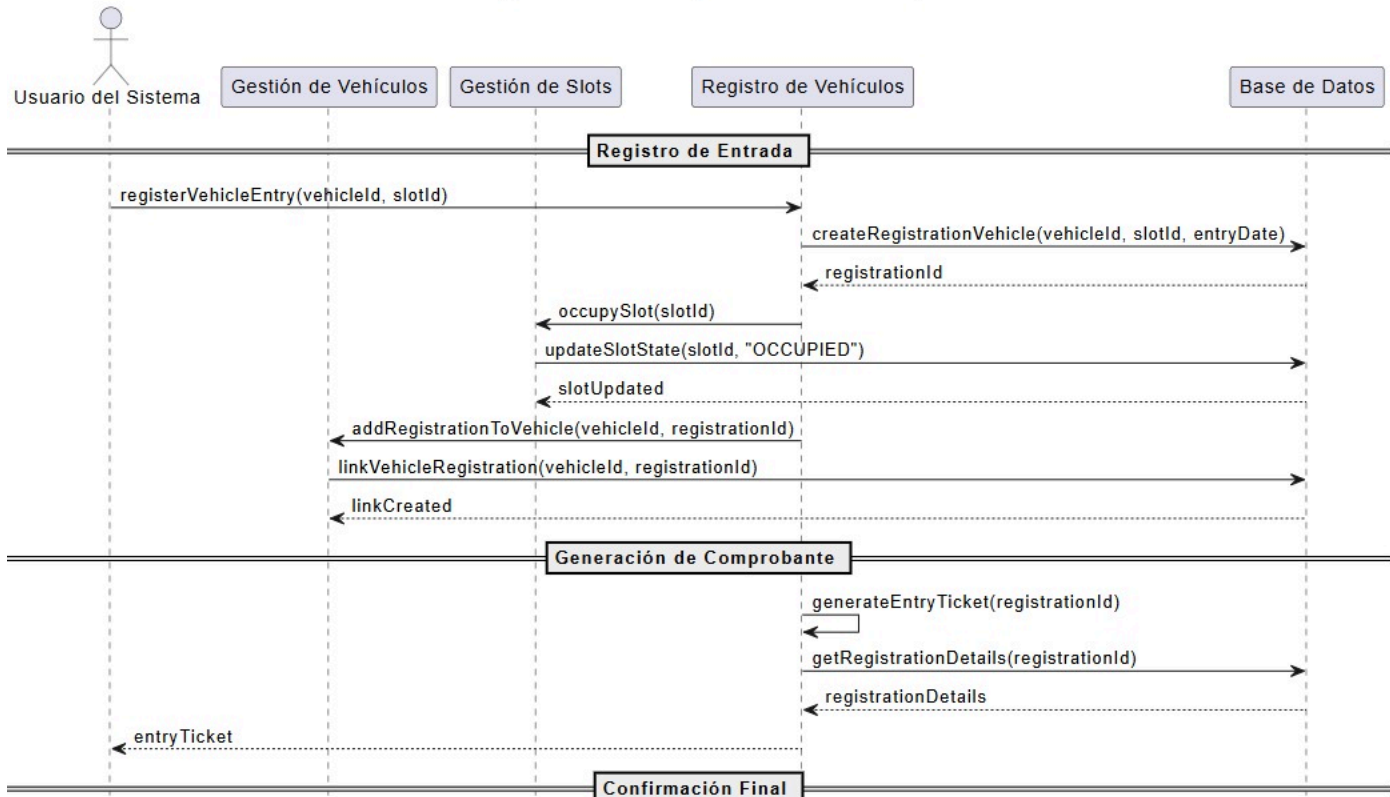
Parte 2: Selección/Registro de Vehículo



Parte 3: Búsqueda de Slot Disponible



Parte 4: Registro de Entrada y Generación de Comprobante



Parte 4: Registro de Entrada y Generación de Comprobante

